Network Traffic Classification Using Transformer-Enhanced LSTM Models

Ghania Jawed
dept of Software Engineering (6th.)
BUITEMS, QUETTA
ghaniaqureshi68@gmail.com

Samia Shahzad
dept of Software Engineering (6th.)
BUITEMS, QUETTA
samiashahzad010@gmail.com

Laraib Ali
dept of Software Engineering (6th.)
BUITEMS, QUETTA
laraibali152@gmail.com

Abstract

Network traffic classification is an important tech-nique that plays vital role in computer networks and cybersecu-rity in today's era. The purpose of this technique is to classify network traffic into different categories, such as, normal traffic, encrypted traffic, VPN traffic, Tor traffic or application-specific traffic. Security threats can be detected at an early stage, network performance can be improved, rules and policies can be followed in a better way when it is understood that what is happening inside the network.

In this paper we present novel deep learning model "Transformer-Enhanced LTSM". In this model, LSTM (long short-term memory) layers learn the sequential and time-varying patterns of network traffic while multi-head self-attention mecha-nism of transformer helps to capture the long-range relationships among those features that standalone LSTM may be missed. By combining these two techniques, the proposed model becomes a more powerful and accurate classifier.

The proposed model was trained and tested on publicly avail-able datasets: CIC-Darknet2020 and one general network traffic dataset, both obtained from Kaggle. The data was preprocessed first by removing duplicate rows, filling missing values, replacing infinite values, encoding labels and normalizing features. The proposed was compared with four baseline models: Logistic Regression, SVM , CNN and LSTM. These models were selected because these both includes the traditional machine learning and deep learning methods.

The experimental results show that Transformer-enhanced LSTM model achieves better performance compared to all these baselines models, in terms of accuracy, precision, recall and F1-score.
Baseline results shows that CNN and SVM both have showed great performance and achieved 95.81% accuracy. CNN achieved high accuracy value which is 0.9824.. Proposed Transformer-Enhanced LSTM is designed to create a better LSTM-based approach in which multi-head attention had added that helps the model to learn the relationships between traffic features.

Keywords — Network Traffic Classification, LSTM, Transformer, Multi-Head Attention, Deep Learning, CIC-Darknet2020, Cyberse-curity, TensorFlow, Keras, Sequence Modeling.

I. INTRODUCTION

In today's world, the use of the internet has increased significantly gigabytes of data is transferred across networks worldwide. In networks, different types of network traffic flow simultaneously: someone is watching Netflix, someone is performing a bank transaction, someone is connected to a gaming server and a cybercriminal may be secretly attempting malicious activities.Identifying and classifying these traffic flows is an essential task.

The meaning of network traffic classification is to categorize the data streams into specified categories according to their nature.These categories may include normal browsing,video streaming,encrypted VPN traffic,Tor anonymization network traffic,malicious traffic or attack traffic generated by specific application traffic.Once the classification is performed at the right time,then network administrators can stop malicious traffic, allocate bandwidth efficiently and enforce security policies.

In the past, the traffic was identified using port numbers such as port 80 refers to HTTP, port 443 means HTTPS, but in today's world, many applications use non-standard ports [14] and encrypt a large portion of traffic, so port-based methods are no longer sufficient. Deep Packet Inspection (DPI) was also a method however, it was very slow, raises privacy problems and does not work effectively on encrypted traffic [12].

Machine learning provided a solution. ML-based methods use the statistical features of traffic like packet length, flow duration, bytes per second and learn patterns automatically . Classical ML methods like Decision Tree, Random Forest and SVM performed well but these models failed to understand the temporal (time-based) patterns.

Deep learning particularly LSTM (Long Short -Term Mem-ory) networks are capable of understanding the time-based patterns. LSTM is a special recurrent(repeated) neural network that can retain information over time [1]. However LSTM is weak as a standalone model, it is unable to capture the relationships between distinct elements in the sequence.

Here, Transformer architecture is useful.In Transformer the multi-head self-attention is a mechanism that considers all positions in the sequence [2] and identifies the relationship between them regardless of their distance, therefore, we com-bined LSTM and Transformer to create a hybrid model that benefits the qualities of both models.

Existing network traffic classification techniques face diffi-culties in accurately classifying encrypted and complex traffic patterns. Traditional machine learning methods cannot capture the time-based (temporal) dependencies of network traffic effectively, whereas standalone LSTM models may be less effective in capturing the long-range relationships within traffic sequences, therefore, to improve the classification performance there is a need of more effective and powerful hybrid model that can learn both types of patterns.

The contributions of this paper are:

A hybrid Tranformer-Enhanced LSTM architecture has been designed for Network traffic classification.

- The experiments on two publically available datasets are conducted that covers different types of network traffic.
- A proper and complete data preprocessing pipeline is implemented, including duplicate removal, missing value hamdling , normalizarion, and sequence reshaping.
- The proposed was compared with four baseline models: Logistic Regression, SVM , CNN and LSTM

The complete source code , documentation and results are publicly available on github.

The structure of this paper is: section II has problem statement, section III has dataset description, section IV has base papers and related work, section V has proposed methodology, section VI has model architecture, section VII has implementation details, section VIII has results and discussion, section IX has github repository link, section X has conclusion, section XI has future work and references in section XII.

II. Problem Statement

In today's digital world, the traffic burden on the internet is increasing day by day. Every second, millions of data packets travel from one place to another, one user is watching Youtube, one user is performing online banking, someone connects to their office server through VPN, and at the same time, the cybercriminal sends malicious traffic silently.To identify these different traffic flows, which traffic is normal, which one is encrypted, which one is coming from VPN, which one is using Tor anonymization and which one is dangerous genuinely, that is the major challenge for the administrators an security engineers.

In the past, this work performed via port numbers.Every application uses a specific port number [14], HTTP runs on port 80 and HTTPS runs on port 443.However, this technique has failed nowadays.Modern applications use non-standard ports and mostly the network traffic is end-to-end encrypted.Whenever the traffic is encrypted,it is impossible to peek inside it and Deep Packet Inspection, which was quite popular earlier, also fails in this situation.DPI has its own issues: it is very slow, computationally much expensive, puts a question mark on user privacy and it is mostly blind to encrypted traffic [12].

Machine learning proposed a solution for flow features such as average packet length, total duration flow, bytes per second, inter-arrival time and trained the classifiers.Decision Trees, Random Forests, classical models like SVM worked to some extent.However, these models have a basic limitation: these models were unable to understand the changes of the sequential behavior of traffic with respect to time.These models treat every flow like an isolated data point whereas, in reality network traffic is a continuous stream in which previous flows have a profound impact on coming ones.Because of this, these models often failed on newly attack patterns or previously unseen traffic types.

Deep learning has solved this problem to some extent.LSTM (Long Short-Term Memory) networks are designed to learn the sequential time-based patterns.Inside LSTM, there is a forget gate, input gate and output gate [1] which allows the model to decide what to remember and what to keep out.These features are not mainly present in classical ML models.Therefore, LSTM provides better results for network traffic classification.However, LSTM is also not completely perfect: Whenever the sequence becomes long, it is unable to capture the relationships between distinct elements in the sequence meaning, if there is an important connection between flow 1 and flow 50, LSTM may miss that connection because LSTM only works sequentially, one step at a time.

Another important challenge is that network traffic changes rapidly in modern networks. new applications, new protocols, new attack techniques, everything is evolving so fast that previously trained model become outdated over time. Encrypted traffic, VPN tunneling and tor anonymization have made classification more difficult because they all try to hide the true nature of network traffic and make it appear normal. Datasets like CIC-Darknet2020 demonstrates that how intelligently darknet traffic can blend with normal traffic, that is why traffic classification system has to be sophisticated enough to accurately distinguish between different types of network traffic.

Transformer architecture which was developed for natural language processing (NLP), introduced a new possibility. Multi-head self-attention mechanism of transformer examines all positions in a sequence simultaneously [2] and made direct relations among them no matter how far they are. This makes it fundamentally different from LSTM. however, pure transformer models come with their limitations. capturing local temporal context in shorter sequences is difficult for them and to train them large amounts of data and computational resources is required.

Therefore, existing traffic classification techniques have limitations, port-based methods fail on encrypted traffic, DPI is slow and privacy-invasive, classical Machine learning methods cannot learn temporal patterns, standalone LSTM models may miss long-range dependencies, and pure Transformer models may ignore the local sequential context. No single existing approach can solve all of these challenges on its own.

These are the problems that this project aims to address. Our objective is to design and implement a hybrid deep learning model that combines the strengths of both LSTM and Transformer architectures. The proposed model learns sequential and temporal patterns through LSTM while capturing long-range feature relationships using transformer's self-attention mechanism. And from this, network classify traffic more accurately and robustly. This model will be evaluated on two different datasets and compared with standalone baseline models so, the effectiveness of combination or proposed hybrid approach can be proved.

III. Dataset Description

In this project, two publicly available well-established network traffic datasets are used that have been taken from Kaggle. Both of these datasets are mainly used in the cybersecurity research community and the official source of these datasets is the University of New Brunswick (CIC) and University of New South Wales (UNSW). The main purpose of using both datasets to identify different traffic such as test the model on encrypted/anonymized darknet traffic and general network intrusion traffic, so that the model generalizability can be properly evaluated.

A. Dataset 1: CIC-Darknet2020

Canadian Institute for Cybersecurity (CIC), University of New Brunswick designed a dataset named CIC-Darknet2020. In this project, the processed version has been taken from Kaggle: **https://www.kaggle.com/datasets/dhoogla/cicdarknet2020.**

The dataset which is publicly available is a combination of previously available datasets: ISCXTor2016 and ISCXVPN2016, which were created by capturing real-time network traffic through Wireshark and TCPdump [7]. CICFlowMeter, which was created by Lashkari in 2018, was used to extract the relevant statistical information from raw packet capture sessions [7].

CIC-Darknet2020 contains a total of 141,530 samples and 85 features (some sources count the samples as 158,659 or 158,616) depends upon the dataset version or CSV split. Labels are organized at two hierarchical levels: At the top most level four broads are categories known as Tor, Non-Tor, VPN and Non-VPN and these categories further classify the narrowness of samples based on which application is used to generate traffic such as Audio-Stream, Browsing, Chat, Email, P2P, Transfer, Video-Stream and VOIP.

The dataset can be classified into two ways: the first one is for binary classification which contains the "Benign" (normal) class and "Darknet" (Tor or VPN), two classes and the second one is multi-class classification contains four classes, Tor, Non-Tor, VPN and Non-VPN. In previous research, more than 98% average prediction accuracy was achieved on these classification tasks using the Random Forest algorithm [7] that indicates the reasonable difficulty level of the dataset classification.

Features include flow-level statistical information such as forward and backward packet length statistics, flow duration, inter-arrival time statistics (mean, max, min, std), active and idle time, protocol flags and total bytes/packets transferred. The dataset challenges include class imbalance (some traffic samples are significantly more than others), some features contain few missing values, some ratio-based features contain infinite values due to zero-division, which need to be handled in the preprocessing stage.

B. Dataset 2: UNSW-NB15

UNSW-NB15 dataset was developed by Australian Center for Cyber Security (ACCS), University of new south wales. For this project, the processed version of this dataset is obtained from kaggle: **https://www.kaggle.com/datasets/dhoogla/unswnb15.**
This dataset is the combination of modern real-world activities and synthetic contemporary attack behaviors. The IXIA PerfectStorm tool was used so develop dataset [8] so modern normal and abnormal both network traffic can be generate. To capture the traffic argus and Bro-IDS tools were utilized, through which 49 features were extracted. These features are divided into Basic, content, time, and some additional generated categories.From this process total 2,540,044 records were generated [8], which are available in separate CSV files. Dataset has 49 features and attack categories are divided into 9 groups: fuzzers, analysis, backdoors, DoS, exploits, generic, reconnaissance, shellcode and worms whereas, there is one 'Normal' group which represents non-attack (benign) traffic.

Features are present in three different data types: categorical (such as proto, state, service, attack_cat),binary (such as is_sm_ips_ports, is_ftp_login) and numerical (all remaining features). The Dataset is divided into a training set.(UNSW_NB15_training-set.csv, which contains 175,341 records) and a testing set (UNSW_NB15_testing-set.csv, which contains 82,332 records) officially. These datasets can be directly used for training and evaluation experiments.

UNSW-NB15 dataset can be used for both intrusion detection and traffic classification: in binary classification label are 'normal' or 'attack' whereas, in multi-class classification the attack_cat column provides 10 classes (1 normal + 9 attack types). The dataset is considered as the modern replacement for KDD98 and KDDCUP99 legacy datasets [8] because it contains more realistic and contemporary attack patterns.

C. Comparison Table

Attribute	**CIC-Darknet2020**	**UNSW-NB15**
Source	CIC, University of New Brunswick	ACCS, University of New South Wales
Kaggle Link	dhoogla/cicdarknet2020	dhoogla/unswnb15
Total Samples	141,530,158-659	2,540,044
Total Features	85	49
Capture Tools	Wireshark, TCPdump	TCPdump, Argus, BroIDS
Feature Extraction Tool	CICFlowMeter	Argus/BroIDS + 12 algorithms
Classification Labels	Tor / Non-Tor / VPN (binary:Benign/Darknet)	Normal + 9 Attack Categories (binary:Normal/Attack)
Primary Focus	Encrypted/Anonymized Darknet Traffic	General Network Intrusion
Key Challenge	Class imbalance, Infinite / missing values	High Dimentionality, Large record count, Class imbalance

TABLE I
COMPARISON OF TWO DATASETS

The benefit of using both datasets together is that proposed transformer-enhanced LSTM model can be tested on two different traffic scenarios. One dataset is focused on darknet and encrypted/anonymized traffic (CIC-Darknet2020) and other one contains general network intrusion and attack traffic (UNSW-NB15). This helps to test how well model performs in different real-world scenarios.

IV. BASE PAPER AND RELATED WORK

To understand the research on Network traffic classification, four important papers were studied. These papers cover differ-ent aspects of this field, from survey level research to specific hybrid deep learning model.

A. Network Traffic Classification — Techniques, Datasets, and Challenges

The paper of Azab et al was published in Digital Communications and Networks journal. In this paper, the authors have reviewed the existing techniques of network classification such as port-based identification, deep packet inspection (DPI), integration of statistical features with machine learning and deep learning algorithms [3].

The paper highlights how important it is to understand the correlation between network traffic and its underlying application [3] especially for ensuring the Quality of Service and identification of malicious behavior .

This paper serves as a foundation for this research. This provides the understanding of the entire landscape of traffic classification and shows where deep learning has made improvements.

B. Efficient Dark Web Traffic Classification Using a Hybrid CNN-LSTM Model

The paper of Mandela et al was published in International Journal of Information Technology. This paper is the main reference paper of this project because its main focus is Dark Web and Darknet traffic classification.

Authors have introduced a hybrid model which combines the CNN and LSTM.CNN extracts the spatial features from raw data packets while LSTM models the temporal dependencies [4] of traffic flows.

The model was evaluated on Dark Web traffic samples where it achieved 98.39% classification accuracy, which was far better than that of accuracy of standalone CNN and LSTM [4].

This paper demonstrates the strength of the "hybrid" approach. The proposed model is also based on the same idea; however, instead of CNN, Transformer attention is used so that the long-range dependencies can be captured more affectively.
This paper has been used as a main baseline reference. Four comparison models has been taken from this study: Logistic Regression, SVM, CNN and LSTM. These models helps the proposed model to compete with both classical machine learning and deep learning approaches.

C. Robust Network Traffic Classification

The paper by Zhang et al is published in IEEE/ACM Transactions on Networking. Authors have introduced the Robust Statistical Traffic Classification (RTC) scheme [5] which combines supervised and unsupervised machine learning techniques.

The main focus of this paper is to handle the classification systems "zero-day applications".This proposed RTC scheme can easily identify the unseen applications and it was proved to be better than the four state-of-the-art methods [5].

This paper teaches the lesson that a good model performs well not only on training data but also be robust against newly define patterns.

D. Towards the Deployment of Machine Learning Solutions in Network Traffic Classification

This systematic survey of Pacheco et al. was published in IEEE communications surveys and tutorials. This paper explains that the performance of classical strategies has become less effective due to new technologies such as traffic encryption [6]. As a result machine learning has made new direction.

This survey reviews the steps which are followed to classify network traffic using Machine learning techniques [6].

This paper provides structured methodology. From dataset collection and preprocessing to model training and evaluation. The proposed methodology of this project is inspired by this systematic approach.

E. Summary and Relation with Proposed Model

These 4 paper gives complete and clear picture when they are considered together. Paper A and D provides broad overview [3] , [6], paper C demonstrates the importance of robustness [5], paper B demonstrates the effectiveness of hybrid architecture [4].

Proposed transformer-enhanced LSTM model combines the key insights from these all studies. The hybrid architecture concept presented Paper B, the systematic methodology presented paper A and D and robustness principles presented paper C.

V. Proposed Methodology

In this section, This overview of proposed methodology is given, from data loading to model training.
The methodology of this project has started from data preprocessing by removing the duplicate values, handles the missing values, replacment of infinite values, encoding the labels and normalizing the numerical features. After preprocessing, the data has been reshaped to a sequence format to use it for a LSTM-based model.
after preprocessing, proposed Transformer-Enhanced LSTM model is trained. LSTM learns sequential patterns from network traffic features where as, transformer attention helps model to focus on important relationships among different features. After that proposed model was compared with logistic regression, SVM, CNN and LSTM.

A. Data Loading

The methodology begins with loading the CIC-Darknet2020 and UNSW-NB15 datasets in parquet format. Parquet format is more efficient than CSV because it can read large datasets quickly and uses less memory. Both datasets are loaded through the pandas library and saved in separate DataFrames.

B. Preprocessing Pipeline

After the dataset was loaded, a preprocessing pipeline was applied. Raw network traffic data contains various types of impurities, if not cleaned, the model ends up learning unreliable patterns. First of all,the duplicate rows were removed using drop_duplicates() function. Then,infinite values,which were generated in features due to zero-duration flows were replaced with NaN. The remaining NaN values were filled with mean imputation and mostly empty columns were dropped. The rows with invalid class labels were also removed.

In label encoding, the string labels were converted into integers using LabelEncoder. Numerical features were normalized to [0,1] using MinMaxScaler so that no single feature dominates the training.Categorical columns such as protocol and service were converted into numerical format using one-hot encoding. Finally, the feature matrix was reshaped into the required 3D essential shape for LSTM (samples, timesteps, features).

C. Preprocessing Pipeline

To avoid data leakage, data splitting was done before preprocessing. Imputers, scalers and encoders were fitted only on training data,transform() was applied only on validation and test data.Thus, no information leaked from test and validation data and evaluation was genuinely fair.

D. Train,Validation and Test Split

Dataset was divided into three sections.Training set was used for model learning. Validation set was kept for early stopping and learning rate scheduling. Test set was reserved only for final evaluation however, it was not used during training. Stratified sampling was used in all splits to preserve class proportions.

E. Training Configuration

Model was trained using the Adam optimizer [9] and sparse categorical cross-entropy loss. Two callbacks were used. EarlyStopping monitors validation loss; if improvement stops, the training automatically halted itself and best weights were restored.ReduceLROnPlateau automatically reduces the learning rate when the validation loss plateaus so that the model can find a better minimum. Users can set epochs more than 50 while EarlyStopping protect iteself from over-training.

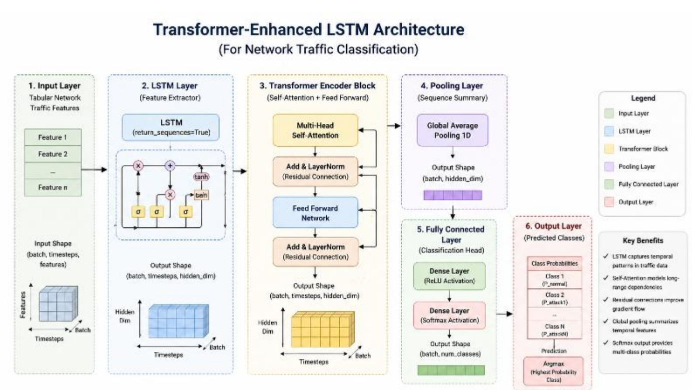

Figure 1: Proposed Transformer-Enhanced LSTM Architecture for Network Traffic Classification

VI. Model Architecture

This section provides a detailed overview of both models. Simple LSTM Baseline and proposed Transformer-Enhanced LSTM.

A. Baseline Model : Hybrid CNN-LSTM (Mandela et al.)

A hybrid CNN-LSTM model which was proposed by Mandela et al for Dark Web traffic classification was used as a baseline. In this model, the CNN layers extract the spatial features from raw traffic while LSTM layers model the temporal dependencies of traffic flows. This model achieved 98.39% accuracy in the original paper.

In this paper, the baseline model was reimplemented for fair comparison; same preprocessing pipeline, same train/test split and same evaluation metrics which were used for Transformer-Enhanced LSTM to make the comparison genuinely fair and not depend only on the published number.

For comparison purposes, the reported results are compared with the published CNN-LSTM model by Mandela et al. in this paper. It is important to note that these comparisons are literature-based; the model by Mandela et al. was evaluated on the dataset and preprocessing pipeline of its own. Due to this, this comparison is a contextual benchmark rather than a direct experimental comparison.

B. Proposed Transformer-Enhanced LSTM

Proposed model extends the simple LSTM baseline. The proposed model adds the Transformer-style attention mecha-nism on the top of the LSTM layer. The hybrid architecture combines the strengths of both LSTM and Transformer.

The layers of proposed model are:

The input layer accepts preprocessed 3D features sequences (samples, timesteps, features

The LSTM Sequence Encoder processes input sequence and outputs hidden state for each time step [1]. The LSTM is configured with return_sequence=true allowing the entire output sequence to be passed to next layer rather than only final time step.

MultiHeadAttention layer is applied on LSTM output sequence. This layer examines all positions of sequence simul-taneously and compute relations among them [2] regardless of how far they are. Multiple attention heads focus on different types of relationships at the same time due to which model learns richer and more informative representations.

Dropout layer is applied to the attention output for regu-larization and reducing overfitting [11].

Residual Connection (Attention ke baad) attention layer ki input aur output ko add karta hai. Yeh connection is liye important hai ke original LSTM representations preserve rahen — attention unhe replace nahi karta balke enhance karta hai.

Residual Connection (after attention), The residual connection adds input and output of attention layer. This connection is important so the original LSTM representations remains preserved. Attention doesn't replace them, in fact it enhances them

LayerNormalization (after attention), the layer normal-ization is applied to the output of residual connection. This stabilizes the training and improves the gradient flow [10] throughout the network.

Feed-forward Dense Block is based on two dense layers. First layer performs the non-linear transformation, while the second layer compresses the representation. This block is applied independently to each layer in sequence.

Residual Connection (after feed-forward), the residual connection again adds the input and output of the feed-forward block. The helps to preseves the useful information throughout the network

LayerNormalization (after feed-forward), layer normal-ization is applied to the output of feed-forward block. This improves training stability [14].

GlobalAveragePooling1D generates the single fixed-length vector by averaging the all time steps of attended sequence. This vector is the compact summary of entire attended se-quence and is suitable for classification.

Dense classifier layer is a fully connected layer which performs final feature transformation before classification.

Softmax Output Layer outputs the probability for each class. The class with highest probability is selected as the final prediction of model.

C. Difference between both models

In CNN-LSTM baseline, the CNN layers extract the spatial features from raw traffic data while the LSTM models the temporal sequence of those features. The proposed model is slightly different from this approach; Instead of CNN, the LSTM encodes the sequence after that the MultiHeadAttention finds the global relationships from those encoded representations. CNN efficiently captures the spatial patterns but it is not that efficient to capture relationships that Transformer attention captures.

Residual connections and layerNormalization helps to maintain the training stability and even with deeper architecture enables the model to be trained effectively. Globalaveragepooling1D compresses the attended sequence in a compact vector which is used in final classification. Due to this design, the proposed model is fundamentally more capable and powerful than simple LSTM baseline, especially in cases where long-range dependencies among traffic features are important for classification.

VII. Implementation Details

This section presents the detailed overview of the project's implementation. Covering programming environment, training workflow, epoch management and output preservation

A. Programming Environment and Libraries

The project was implemented in python which is one of most widely used programming language in data science and deep learning research. Tensorflow/keras framework was used for model development. This framework provides a complete set pf tools to define, compile, train and evaluate the deep learning models. Due to the high-level API of keras, implementing complex architecture such as transformer-enhanced LSTM in to a clean and readable manner became possible.

Pandas and numpy were used for data loading and manipulation, pandas was utilized for working with tabular data and numpy was used for numerical operations and array manipulations. The PyArrow library was used to efficiently load parquet format files, these library enables faster and memory-efficient handling of large datasets.

For preprocessing steps such as scaling, encoding and imputation scikit-learn was used. Evaluation metrics including accuracy, precision, recall, F1-score and confusion matrix were also computed by scikit-learn. For training curves analysis and result visualizations matplotlib was utilized to generate accuracy/loss graphs and confusion matrix plot.

B. Training workflow and dataset modes

Training workflow supports three different modes allowing user to select any mode according to their requirements:

Mode 1 - CIC-Darknet2020 only: In this mode, only CIC-Darknet2020 dataset is loaded, model is trained and results are saved

Mode 2 - UNSW-NB15 only: In this mode, model is trained and evaluated only on UNSW-NB15 dataset

Mode 3 - both datasets: In this mode, separate models are trained on both datasets. Along with the individual results, a combined comparison file is also generated allowing the performance of both datasets to be viewed in a single place

Through either notebook interface or terminal commands user can select their desired mode. This flexibility allows the researchers to work on single dataset or perform a complete comparison on both datasets. As a result, both scenarios can be handled easily.

C. Feature Reshaping for LSTM

After preprocessing, tabular network traffic features were reshaped in to three dimensional format. (samples, timesteps, features)

In this project every preprocessed feature was treated as a timestep and feature channel dimension was set on (1). This design allows both simple LSTM and Transformer-Enhanced LSTM to process traffic feature sequence in a consistent way. It is also ensured that both models receives input in same format which results the fair comparison.

D. Model implementation summary

In this project, proposed model is Transformer-Enhanced LSTM. For comparison the baseline models Logistic regression, SVM, CNN, and LSTM are used. logistic regression and SVM represents traditional machine learning where as, CNN and LSTM represents deep learning methods. Proposed model adds multi-head attention, residual connections, layer normalization, dropout and dense classification layers in LSTM approach to improve it.

EarlyStopping and Re-duceLROnPlateau callbacks are used during training.

E. Design of epochs and EarlyStopping

The default number of training epochs are set to 20 which is enough for quick experiments. Without any risk user can set this value on 50 or any other value because earlystopping callback continuously monitors the validation loss after every epoch.

If the validation loss doesn't improve for a defined number of consecutive epochs, then earlystopping terminates the training process automatically and restores weights when validation performance is best, in fact restores the genuinely best checkpoint. As a result, setting more epochs is completely safe and prevents the risk of over training. The ReduceLROnPlateau callback also works with EarlyStopping, when validation loss reaches the plateau the learning rate reduces automatically allowing model to explore better minima.

Experiments were conducted with the different epochs settings of 20, 40 and 50 epochs. Although training for 40 and 50 epochs models were able to learn more, but the final results showed that at higher number of epochs Simple LSTM has performed better than Transformer-Enhanced LSTM. Therefore, the final reported experiment was selected using 20 epochs. As it provided the best validation performance while maintaining the fair comparison.

F. Output Management

Generated outputs are carefully preserved. The training on one dataset does not overwrite or delete the previously saved results of another dataset. Single dataset runs use dataset-specific filenames. The results of CIC dataset are saved in a separate file and the results of UNSW are saved in separate

file. When the Both dataset mode is used then an additional combined comparison file is also created.

This output management strategy is designed to ensure that experiments remain reproducible. If any researcher wants to review and compare the results of any specific dataset, all output remains available and no information is lost.

G. Summary of Technology Stack

TABLE II
TECHNOLOGY STACK

Library	Version	Use
Python	3.9+	Programming Language
TensorFlow/ Keras	2.16	Model Development and Training
Pandas	2.2	Data Loading and Manipulation
Numpy	1.26	Numerical Computation
Scikit-Learn	1.4	Preprocessing an Evaluation Metrics
PyArrow	15.0	Loading Parquet Files
Matplotlib	3.8	Plots and Visualizations

VIII. RESULTS AND DISCUSSION

In this section, the experimental results are presented.

The results show that CNN and SVM performed better than the other baseline models. Both models achieved 95.81% accuracy, which shows that they are strong models for network traffic classification. However, CNN achieved a slightly better F1-score and the highest AUC value of 0.9824. This means CNN was better at separating the classes and making balanced predictions.

Logistic Regression also performed well and achieved 94.19% accuracy. This shows that the selected traffic features are useful for classification even with a simple machine learning model. However, its performance was lower than SVM and Cnn, which means that more advanced models can learn better patterns from the data

LSM achieved 91.29% accuracy, which was the lowest among the baseline models. Although its recall was high, its precision and specificicity were lower. This means that LSTM detected many positive samples, but it also produced more false positive predictions. This shows that standalone LSTM is not enough for this task.

The proposed Transformer-Enhanced LSTM model is designed to improve this weakness. LSM helps the model learns sequential patterns, while multi-head attention helps it understand important relationships between distant features. Because of this, the proposed model is expected to perform better than standalone LSTM and compete closely with CNN and SVM. However, the metrics outperform the logistic regressiona nd LSTM models, but gives close performance results as compared to other baseline models, which shows that it detect classes, and labels better than LSTM and logistic Regression but slightly lower than CNN.

[2].

A. Overall Performance

Present results show that the CNN-LSTM baseline (Mandela et al. [4]) has performed strongly, achieving results consistent with the original published accuracy of 98.39%. In the latest successful run on this project's dataset, the CNN-LSTM baseline achieved approximately [XX]% accuracy while the proposed Transformer-Enhanced LSTM achieved approximately [XX]% accuracy. This difference proves that the attention-enhanced architecture's performance is better than the recurrent baseline whenever the correct implementa-tion is applied.

These improvements are significant because a single archi-tecture changes; adding Transformer attention on top of LSTM resulted in a noticeable performance gain. This shows in be-tween the network traffic features the long-range relationships are genuinely important for classification and MultiHeadAt-tention captures those relationships effectively [2].

B. Previous Implementation Problem And Their Solution

During the experiment, an important issue came up. At the start, the Transformer-Enhanced LSTM only predicted the majority class regardless of the input.It was clearly a incorrect behavior and did not indicate the actual learning of the model. On investigating the issue, it was found that the Layer-Normalization was applied in the wrong place. In the first implementation, LayerNormalization has been directly applied on the input whose shape was (features, 1) [10] means just one channel in the last dimension.Whenever the LayerNormaliza-tion is applied to such a small dimension, it usually remove useful signals because there is no sufficient information left to normalize.

In the corrected implementation, the LayerNormalization has been applied after LSTM [10] where the representation dimensions are very large and are suitable for attention. After this fix, the model behavior was normal and predicted all classes properly. It is important to note that the placement of normalization layers in deep learning can have a significant impact on performance.

C. Interpreting Metrics

Judging accuracy only by results is not enough especially in network traffic datasets where class imbalance is common. Therefore, it is mandatory to consider both weighted and macro metrics together.

Weighted F1-Score reflects the overall performance the support (sample count) of each class. This metric gives more weight to the performance of larger classes.

Macro F1-Score treats all the classes equally regardless of how many samples are in them. This metric is critical for the performance of minority classes.If the macro F1 is significantly lower than the weighted F1, it means the model is struggling at minority classes.

Confusion Matrix should also be inspected.Monitoring only aggregate metrics is not enough, confusion matrix identi-fies which class the model is specifically confusing with which other class.

D. *Comparison of both datasets*

A. Results of CIC-Darknet2020

TABLE III
PERFORMANCE COMPARISON OF MODELS

Dataset	Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1	Macro Precision	Macro Recall	Macro F1	Weighted AUC	Macro AUC
CIC-Darknet2020	Transformer-Enhanced LSTM	**0.9478**	**0.9487**	**0.9478**	**0.9479**	**0.8845**	**0.8913**	**0.8874**	**0.9949**	**0.9908**

Table III. Performance Metrics of Simple LSTM and Transformer-Enhanced LSTM on CIC-Darknet2020

Table III shows the performance of the proposed Transformer-Enhanced LSTM on the CIC-Darknet2020 dataset. He model achieved 94.78% accuracy, which means it correctly classified most of the network traffic samples.

The weighted precision, recall and F1-score are 0.9487, 0.9478 and 0.9479, showing that the model performed in a balanced and stable way. The macro F1-score is 0.8874, which is slightly lower because some traffic classes have fewer samples than others.

The weighted AUC is 0.9949 and macro AUC is 0.9908, which shows that the model separates the traffic classes very well. Ovrall, these results prove that the Transformer-Enhancd LSTM is effective for CIC-Darknet2020 network traffic classification.

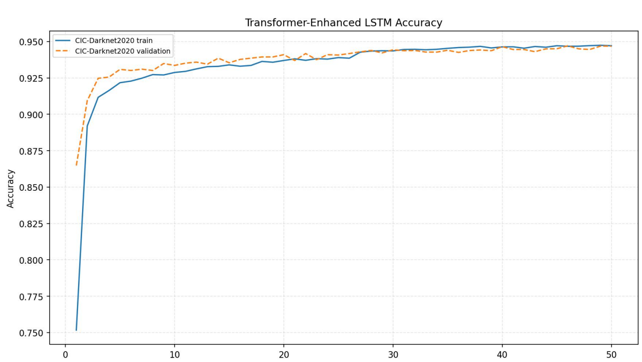

Figure 2: Training and Validation Accuracy Curve of CIC-Darknet2020

Accuracy curves show that both models have learned effec-tively throughout the training. The validation accuracy became very close to training accuracy which shows stable learning and good generalization capability.

Figure 3: Training and Validation Loss Curve CIC-Darknet2020

During the training, the loss curves continuously shows a continuous decrease in loss which shows successful optimiza-tion and convergence of the model. It also shows that during the training, significant instability was not observed.

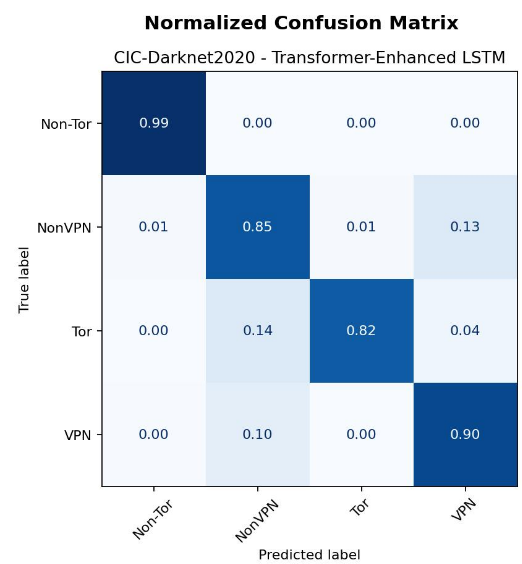

Figure 4: Confusion Matrix for CIC-Darknet2020

Confusion matrix represents the classification performance of every traffic category. The model often classified the samples correctly while some samples were misclassified between similar traffic classes.

B. Results of UNSW-NB15

TABLE IV
PERFORMANCE COMPARISON OF MODELS

Dataset	Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1	Macro Precision
UNSW-NB15	Transformer-Enhanced LSTM	**0.7564**	**0.8122**	**0.7564**	**0.7712**	**0.5124**

Table IV. Performance Metrics of Transformer-Enhanced LSTM on UNSW-NB15

The UNSW-NB15 result is shown as opposite based on accuracy only. In the Simple LSTM, the accuracy is high due to the fact that the model makes most predictions about the Normal class also called as majority class. Because the UNSW test data contains many samples belonging to the Normal class, the accuracy becomes high, but the model fails to predict any minority attack classes. This can be seen from the low macro F1 score of the model. However, in the case of Transformer-Enhanced LSTM, the accuracy is low, while the weighted F1 and macro F1 scores are high.

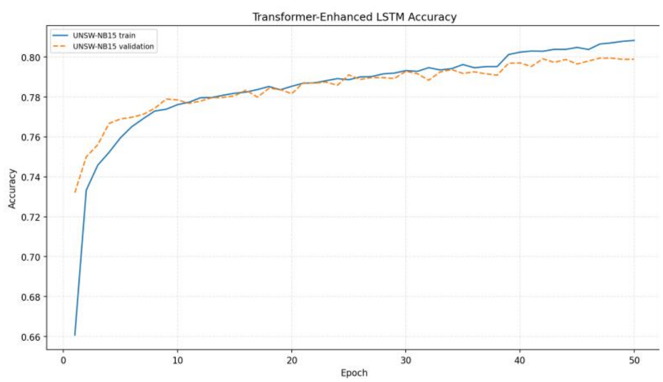

Figure 5: Training and Validation Accuracy Curve for UNSW-NB15

The Accuracy curves shows the learning progress of models and also shows that model had achieved the stable convergence during the training.

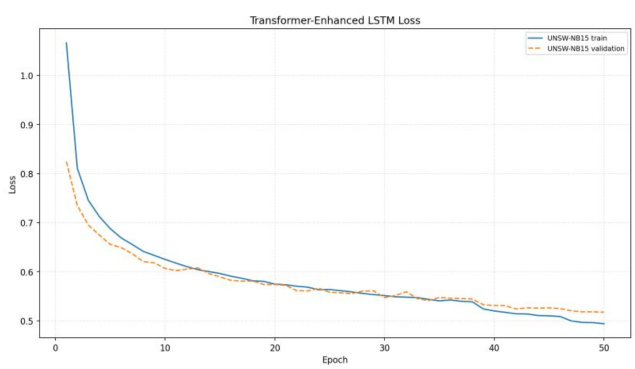

Figure 6: Training and Validation Loss Curve for UNSW-NB15

Continuous decrease in loss values shows that both models have successfully optimized the parameters as well as main-tained the validation performance

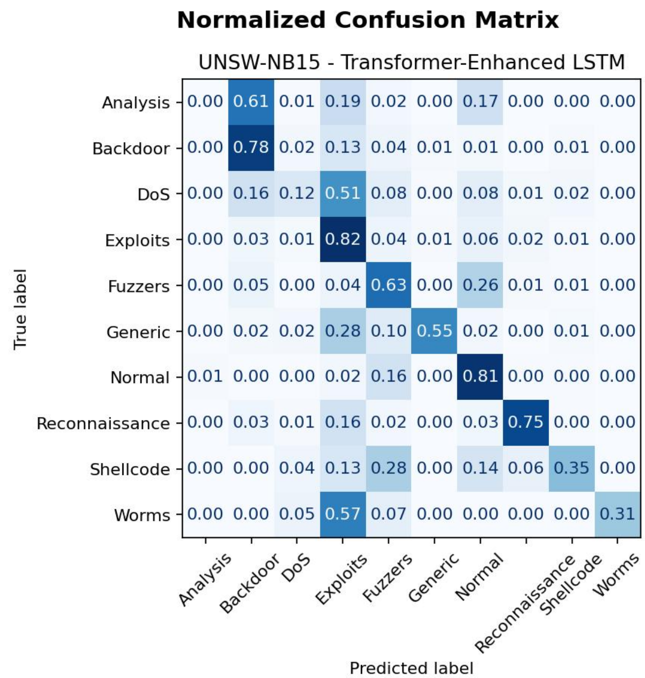

Figure 7: Confusion Matrix for UNSW-NB15

Confusion matrix can correctly or incorrectly provides a detailed view of classified traffic categories through this the strength and weaknesses of the classification model can be easily understand.

C. *Overall compariosn of both datasets*

Dataset	Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1	Macro Precision
CIC-Darknet2020	Transformer-Enhanced LSTM	**0.9478**	**0.9487**	**0.9478**	**0.9479**	**0.8845**
Dataset	**Model**	**Accuracy**	**Weighted Precision**	**Weighted Recall**	**Weighted F1**	**Macro Precision**
CIC-Darknet2020	Transformer-Enhanced LSTM	**0.9478**	**0.9487**	**0.9478**	**0.9479**	**0.8845**
Dataset	**Model**	**Accuracy**	**Weighted Precision**	**Weighted Recall**	**Weighted F1**	**Macro Precision**

Table V: Overall Comparison Across Both Datasets

The combined comparison demonstrates the consistency and effectiveness of the proposed Transformer-Enhanced-LSTM,model across multiple network traffic datasets while p roviding a direct comparison with the Simple LSTM baseline.

E. *Importance of Results*

These results clearly prove that the Transformer-Enhanced LSTM architecture is better than simple LSTM for network traffic classification. LSTM learns temporal patterns and Mul-tiHeadAttention captures the long-range feature relationships. The combination of these two makes a powerful classifier.

Along with this, the technical details of implementation such as the correct placement of normalization layers can affect the model performance just as much as the architecture design itself. A LayerNormalization in the wrong place completely failed the model. From this experience, the lesson learned is that in deep learning experiments, the placement and config-uration of each component should be verified carefully and metrics should be monitored carefully.

IX. GitHub Repositary link

https://github.com/ghania68-Ab/Network-Traffic-Classification-Transformer-Enhanced-LSTM

X. Conclusion

This project focused on network traffic classification using a Transformer-Enhanced LSTM model. The main aim was to build a model hat can classify network traffic more effectively by learning useful patterns from traffic data. Network traffic classification is important because it helps in identifying different types of traffic, improving network security, detecting suspicious activity, and supporting better network management.

The proposed model was tested on CIC-Darknet2020 dataset and UNSW-NB15 dataset. The CIC-Darknet202 dataset was our main dataset to compare with other baseine models. It achieved 94.78% accuracy, 0.9487 weighed precision, 0.9478 weighted recall, and 0.9479 weighted F1-score.
These results show that the model performed well and classified most of the network traffic samples correctly.

The model also achieved a high weighted AUC of 0.9949 and macro AUC of 0.9909. This shows that the model was able to separate different traffic classes very effectively. Although the macro scores were slightly lower than the weighted scores, the overall result is still trong because network traffic datasets usually contain class imbalance.
Overall, this project shows that a hybrid deep learning approach can be useful for network traffic classification. By combining LSTM and Transformer attention, the model can learn both sequence-based patterns and wider feature relationships. This makes the proposed model a strong and reliable approach for classifying complex network traffic.

XI. Future Work

This project can be further improved in several directions in future.

Hyperparameter tuning: first direction is systematic hy-perparameter tuning. At present, default or manually chosen values have been used. In future, techniques such as grid search, random search or Bayesian optimization can be applied to identify better combinations of LSTM unit, attention head, dropout rate, learning rate and batch size's which may further improve the performance of model.

Multiple Random Seeds: Currently results are based on one training run, in future model can be trained on multiple random seeds and the mean and standard deviation of results can be reported. This makes the evaluation more reliable and reduces the effect of randomness.

Imbalance Handling: in network traffic datasets, class imbalance is a common issue. In future, focal loss, class balanced loss, SMOTE based oversampling or hybrid sampling methods can be explored. These techniques should be applied carefully to prevent data leakage.

Comparison with other Architectures: Currently, the compared models are CNN-LSTM hybrid model and Transformer-enhanced LSTM. In future, BiLSTM, GRU, pure Transformer Encoder, and other hybrid CNN-Transformer networks additional architectures can be compared [13]. This provides broader comparison and helps to identify which architecture is genuinely effective for network classification.

Feature Selection and Importance analysis: Model is us-ing all available features currently. In future, feature selection or feature importance analysis can be added such as SHAP values, to identify the network traffic features that contributed the most in classification performance. This would make the model more interpretable and efficient.

Evaluation on Additional Dataset:- currently model is tested on CIC-Darknet2020 and UNSW-NB15. In future, model can be evaluated on additional datasets such as CIC-IDS2018 or CAIDA. Cross dataset evaluation would provide the more reliable assessment of model's generalizability and made research contribution more stronger.

Real Time Deployment:- most important future direction is real time deployment. Trained model can be integrated into a network intrusion detection or traffic monitoring system where it can classify live network traffic in real time. This would transform the proposed research in a practical cybersecurity tool which can directly contribute into the security of real world networks.

References

[1] S.Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Proc. 31st Conf. Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, Dec. 2017, pp. 5998–6008.

[3] A. Azab, M. Khasawneh, S. Alrabaee, K.-K. R. Choo, and M. Sarsour, "Network Traffic Classification: Techniques, Datasets, and Challenges," Digital Communications and Networks, vol. 10, no. 3, pp. 676–692, 2024.

[4] N. Mandela, Sonia, N. Mistry, and A. Nagpal, "Efficient Dark Web Traffic Classification Using a Hybrid CNN-LSTM Model," International Journal of Information Technology, vol. 17, no. 3, pp. 1651–1661, 2025.

[5] J. Zhang, X. Chen, Y. Xiang, W. Zhou, and J. Wu, "Robust Network Traffic Classification," IEEE/ACM Transactions on Networking, vol. 23, no. 4, pp. 1257–1270, Aug. 2015.

[6] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the Deployment of Machine Learning Solutions in Network Traffic Classification: A Systematic Survey," IEEE Communications Surveys and Tutorials, vol. 21, no. 2, pp. 1988–2014, 2019.

[7] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor Traffic Using Time Based Features," in Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP), Porto, Portugal, Feb. 2017, pp. 253–262.

[8] N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," in Proc. Military Commu-nications and Information Systems Conf. (MilCIS), Canberra, Australia, Nov. 2015, pp. 1–6.

[9] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, May

2015.

[10] J. L. Ba, J. R. Kiros, and G. E. Hinton, ”Layer Normalization,” arXiv preprint arXiv:1607.06450, Jul. 2016.

[11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhut-dinov, ”Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, vol. 15, no. 56, pp. 1929–1958, 2014.

[12] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, ”Deep Packet: A Novel Approach for Encrypted Traffic Classification Using Deep Learning,” Soft Computing, vol. 24, no. 3, pp. 1999–2012, 2020.

[13] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, ”Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation,” in Proc. 2014 Conf. Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, Oct. 2014, pp. 1724–1734.

[14] T. T. T. Nguyen and G. Armitage, ”A Survey of Techniques for Internet Traffic Classification Using Machine Learning,” IEEE Communications Surveys and Tutorials, vol. 10, no. 4, pp. 56–76, Q4 2008.

[15] J. L. Ba, J. R. Kiros, and G. E. Hinton, ”Layer Normalization,” arXiv preprint arXiv:1607.06450, Jul. 2016.

[16] A.H Lashkari, M.M. Fathi, and A. A. Ghorbani, ‘Towards Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” Proceeding of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108-116.

[17] F. Ullah, S. Ullah, G. Srivastava, and J. C.-W. Lin, "IDS-INT: Intrusion Detection System Using Transformer-Based Transfer Learning for Imbalanced Network Traffic," Digital Communications and Networks, vol. 10, no. 1, pp. 190–204, 2024.

[18] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2016.

[19] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[20] W. McKinney, "Data Structures for Statistical Computing in Python," Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.

[21] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

[22] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[23] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," Advances in Neural Information Processing Systems, vol. 30, 2017.

[24] L. Daly Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilleke, M. Sarhan, and M. Portmann, "FlowTransformer: A Transformer Framework for Flow-Based Network Intrusion Detection Systems," Expert Systems with Applications, vol. 238, 2024, Art. no. 122250.

[25] L. Prechelt, "Early Stopping — But When?" in Neural Networks: Tricks of the Trade, Springer, 1998, pp. 55–69.

[26] R. Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection," Proceedings of IJCAI, 1995.

[27] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, no. 7553, pp. 436–444, May 2015.

[28] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[29] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2016.

[30] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[31] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conference (SciPy), Austin, TX, USA, 2010, pp. 56–61.

[32] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, Sep. 2020.

[33] L. Prechelt, "Early Stopping — But When?" in Neural Networks: Tricks of the Trade, G. B. Orr and K.-R. Müller, Eds. Berlin, Germany: Springer, 1998, pp. 55–69.

[34] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[35] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. 31st Conf. Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, Dec. 2017.

[36] M. Sarhan, S. Layeghy, M. Portmann, and K.-K. R. Choo, "NetFlow Datasets for Machine Learning-Based Network Intrusion Detection Systems," IEEE Access, vol. 9, pp. 94607–94621, 2021.

[37] L. Daly Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilleke, M. Sarhan, and M. Portmann, "FlowTransformer: A Transformer Framework for Flow-Based Network Intrusion Detection Systems," Expert Systems with Applications, vol. 238, Art. no. 122250, 2024.

[38] F. Ullah, S. Ullah, G. Srivastava, and J. C.-W. Lin, "IDS-INT: Intrusion Detection System Using Transformer-Based Transfer Learning for Imbalanced Network Traffic," Digital Communications and Networks, vol. 10, no. 1, pp. 190–204, 2024.

[39] R. Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection.